

پروژه پنجم آزمایشگاه سیستم عامل

مجید سپاهی - 810100163

امیرحسین صباحی - 810101556

پرسش 1: راجع به مفهوم ناحیه مجازی در لینوکس به طور مختصر توضیح داده و آن را با xv6 مقایسه کنید. صفحه های محافظ به چه منظور استفاده میشوند؟

ناحیه حافظه مجازی در لینوکس بخشی از فضای آدرس فرآیند است که معمولاً برای نگاشت های کتابخانه های مشترک (مانند libc) و همچنین نگاشت های فایل های اجرایی استفاده می شود. در معماری های مدرن لینوکس، فضای آدرس فرآیند به صورت ۳ قسمتی اصلی در نظر گرفته می شود: پایین ترین بخش برای کد، داده ها، و هیپ است که توسط کرنل به صورت دینامیک مدیریت می شود؛ بخش میانی (معمولاً نزدیک به KERNBASE در x86 یا نقطه تقسیم در معماری های دیگر) برای نگاشت های کاربر است و ناحیه بالایی برای کرنل رزرو شده است. این تقسیم بندی انعطاف پذیری بالایی در مدیریت منابع و امنیتی فرآیند فراهم می کند، در حالی که در xv6، به دلیل عدم فعال بودن کامل صفحه بندی در حالت پیش فرض اولیه (همانطور که در فایل Notes اشاره شده است که کد اولیه بدون صفحه بندی کار می کند)، مدیریت آدرس ها عمدتاً از طریق بخش بندی (Segmentation) در ابتدا انجام می شد و نقشه برداری مجازی به شکل ساختار یافته لینوکس نبود و ساختار حافظه کاربر ساده تر بود، شامل کد، داده، هیپ و استک که همه در یک ناحیه پیوسته بالای حافظه فیزیکی نگاشت می شدند، مگر اینکه صفحه بندی به صورت دستی فعال شود.

صفحه های محافظت شده (Guard Pages) مکانیسم امنیتی هستند که در فضای آدرس یک برنامه قرار داده می شوند و معمولاً بین بخش های مهم حافظه مانند هیپ و استک قرار می گیرند. این صفحات به گونه ای پیکربندی می شوند که هرگونه دسترسی به آن ها منجر به خطا (مانند خطای Segmentation Fault) شود. هدف اصلی آن ها جلوگیری از سرریز بافر (Buffer Overflow) است؛ برای مثال، اگر استک بیش از حد رشد کند یا هیپ به صورت غیرمجاز دسترسی یابد و از مرزهای مجاز خود عبور کند، اولین دسترسی به این صفحه محافظ، سیستم عامل را مطلع می سازد و برنامه را متوقف می کند. در حالی که xv6 نیز از مفهوم صفحه محافظ برای جداسازی استک/هیپ استفاده می کند (همانطور که در نقشه حافظه در صفحه ۲ ذکر شده است)، در لینوکس

این مفهوم در کنار ساختار پیچیده VMA ها و مکانیزم‌های قوی صفحه بندی برای تضمین ثبات و امنیت بهتر فرآیندها به کار گرفته می‌شود.

پرسش 2: چرا ساختار سلسله مراتبی منجر به کاهش مصرف حافظه می‌گردد؟

ساختار سلسله مراتبی در جدول صفحه بندی (Page Table) مانند آنچه در معماری x86 و لینوکس استفاده می‌شود، با استفاده از جداول چندسطحی (Multilevel Page Tables) باعث کاهش مصرف حافظه برای نگاشت آدرس‌های مجازی می‌گردد زیرا تنها قسمت‌هایی از جدول صفحه بندی که برای آدرس‌های مجازی استفاده شده و فعال هستند (مانند آدرس‌های سطح بالا یا Page Directory Entries که به جداول سطح پایین اشاره می‌کنند) نیاز به تخصیص فضا دارند در حالی که بخش‌های بزرگی از فضای آدرس مجازی که هرگز استفاده نشده‌اند، نیازی به تعریف کامل جداول صفحه بندی سطح پایین ندارند، بدین ترتیب از اتلاف حافظه برای نگاشت فضای آدرس‌های استفاده نشده جلوگیری می‌شود.

پرسش 3: مفهوم هر بیت یک مدخل ۳۲بیتی در هر سطح چیست؟ چه تفاوتی میان آنها وجود دارد؟

بیت‌های موجود در هر مدخل ۳۲ بیتی در سطوح مختلف جدول صفحه دارای معانی متفاوتی هستند؛ در سطوح بالاتر، بیت‌های آدرس به آدرس فیزیکی جدول صفحه بعدی اشاره می‌کنند، در حالی که بیت‌های کنترلی مانند Present (P) و Read/Write (R/W) حفاظت از دسترسی به آن جدول را تعیین می‌کنند، اما در سطح نهایی مدخل‌ها (PTE ها)، بیت‌های آدرس به آدرس فیزیکی نهایی قاب حافظه اشاره می‌کنند و بیت‌های وضعیت اختصاصی مانند Dirty (D) و Accessed (A) که وضعیت استفاده از صفحه فیزیکی را نشان می‌دهند، در این سطح تنظیم می‌شوند، بنابراین تفاوت اصلی در سطح نهایی نگاشت به فریم فیزیکی و مدیریت وضعیت آن است.

پرسش 4: تابع `kalloc` چه نوع حافظه ای تخصیص میدهد؟ (فیزیکی یا مجازی)

در `xv6`، `kalloc` برای تخصیص بلاک‌های حافظه فیزیکی برای کرنل استفاده می‌شود. این توابع مستقیماً با مدیریت قاب‌های فیزیکی (`Page Frames`) درگیر هستند و نگاشت‌های صفحه بندی (`Page Tables`) برای این حافظه فیزیکی را بعداً توسط کرنل تنظیم می‌کنند تا برای کرنل قابل دسترسی به صورت مجازی شوند. هدف اصلی `kalloc` در سطح پایین، فراهم کردن فریم‌های فیزیکی خام است.

پرسش 5: تابع `mappages` چه کاربردی دارد؟

تابع `mappages` در `xv6` برای نگاشت یک محدوده از آدرس‌های مجازی به فریم‌های فیزیکی معین استفاده می‌شود. کاربرد اصلی آن این است که یک بلوک از آدرس‌های مجازی را در فضای آدرس یک فرآیند به فریم‌های فیزیکی که از قبل تخصیص داده شده‌اند (مثلاً توسط `allocvm` یا در هنگام فراخوانی سیستمی مانند `exec`) مرتبط سازد. این تابع جداول صفحه را به روز می‌کند تا مشخص شود کدام آدرس مجازی باید به کدام آدرس فیزیکی اشاره کند، و معمولاً بیت‌های دسترسی (مانند خواندن/نوشتن و وضعیت `Present`) را نیز تنظیم می‌کند.

پرسش 6:

آیا تمام قسمت‌های حافظه مجازی از آدرس صفر تا `proc->sz` از طریق کاربر قابل استفاده اند؟

خیر، تمام قسمت‌های حافظه مجازی از آدرس صفر تا `proc->sz` در `xv6` لزوماً توسط کاربر قابل استفاده (قابل دسترسی) نیستند. آدرس مجازی در فضاهای آدرس بسیاری از سیستم‌ها از صفر شروع می‌شود، اما در `xv6` و معماری‌های مشابه، معمولاً یک ناحیه رزرو شده در نزدیکی آدرس صفر وجود دارد که برای حفاظت از پشته کاربر (`User Stack`) استفاده می‌شود.

کد exec نشان می‌دهد که دو صفحه در بالاترین حد حافظه کاربر (نزدیک `sz<-proc`) تخصیص داده شده است. صفحه اول از این دو صفحه (که به آدرس `sz - 2*PGSIZE` نگاشت شده است) عمداً از طریق تابع `clearpteu` غیرقابل دسترسی (فقط برای هسته یا غیرقابل دسترسی) تنظیم می‌شود تا یک صفحه محافظ (Guard Page) باشد. دسترسی به این صفحه باعث ایجاد وقفه صفحه (Page Fault) می‌شود و از سرریز شدن پشته (Stack Overflow) به سمت کد یا داده‌های پایین‌تر جلوگیری می‌کند. بنابراین، ناحیه بین آدرس صفر تا آدرس پشته و همچنین صفحه محافظ در بالای ناحیه استفاده شده توسط کد و داده، توسط کاربر قابل دسترسی نخواهند بود یا دسترسی به آن‌ها با محدودیت‌های حفاظتی (فقط برای کرنل) همراه است.

پرسش 7:

راجع به تابع `walkpgdir` توضیح دهید. این تابع چه عمل سخت افزاری را شبیه سازی میکند؟

تابع `walkpgdir` یک عمل کلیدی در مدیریت حافظه مجازی کرنل (مانند xv6) را شبیه‌سازی می‌کند: پیمایش سلسله مراتبی جدول صفحه برای دسترسی به مدخل یک صفحه یا تخصیص جداول میانی.

این تابع آدرس مجازی مورد نظر را می‌گیرد و با استفاده از نشانگر جدول صفحه سطح بالا (که معمولاً در ساختار `proc` ذخیره شده)، در جداول صفحه (Page Directory و Page Tables) به صورت پایین رونده (از سطح بالا به پایین) جستجو می‌کند. این فرآیند، معادل مراحل سخت‌افزاری است که واحد مدیریت حافظه (MMU) هنگام تبدیل یک آدرس مجازی به آدرس فیزیکی طی می‌کند.

بنابراین، `walkpgdir` عمل ترجمه آدرس (Address Translation) سخت‌افزاری MMU را به صورت نرم‌افزاری شبیه‌سازی می‌کند تا کرنل بتواند قبل از وقوع خطای صفحه (Page Fault)، جداول لازم را ایجاد کند یا وضعیت بیت‌های موجود در مدخل‌های جدول صفحه (PTE/PDE) را بررسی و تغییر دهد.

توابع `mmap` و `allocvm` که در ارتباط با حافظه ی مجازی هستند را توضیح دهید.

تابع `allocvm` مسئول گسترش فضای آدرس مجازی فرآیند جاری از اندازه `oldsz` به `newsz` و تخصیص فریم‌های فیزیکی لازم برای صفحات جدید است. هنگامی که کرنل نیاز دارد کد یا داده‌های برنامه را بارگذاری کند یا پشته کاربر را تنظیم نماید، از این تابع فراخوانی می‌کند. `allocvm` ابتدا بررسی می‌کند که آیا فضای آدرس جدید (که توسط `newsz` مشخص می‌شود) با فضای فیزیکی موجود همخوانی دارد یا خیر. اگر نیاز به تخصیص فریم‌های فیزیکی جدید باشد، این فریم‌ها را از طریق مدیریت حافظه فیزیکی کرنل (مانند استفاده از `kalloc` و سپس نگاشت آن‌ها) رزرو کرده و مدخل‌های مربوطه را در جدول صفحه (`pgdir`) فرآیند تنظیم می‌کند تا از بیت `Present` اطمینان حاصل شود و سطح دسترسی مناسبی به آن‌ها اختصاص داده شود.

تابع `mmap` عملیات سطح پایین‌تر نگاشت را انجام می‌دهد و اساساً همان کاری را می‌کند که `MMU` هنگام پردازش یک خطای صفحه برای یک ناحیه حافظه انجام می‌دهد. `mmap` یک محدوده از آدرس‌های مجازی (`va` تا `va + size`) را به محدوده مشخصی از فریم‌های فیزیکی (شروع از `pa`) نگاشت می‌کند و سطح دسترسی‌های مورد نظر (`perm`) را برای آن مدخل‌ها (`PTE/PDE`) تنظیم می‌نماید. این تابع به طور مستقیم توسط توابعی مانند `allocvm` فراخوانی می‌شود تا اطمینان حاصل شود که آدرس‌های مجازی مورد نیاز فرآیند، به فریم‌های فیزیکی معین شده در حافظه اصلی مرتبط می‌شوند و بیت‌های حفاظتی و کنترلی (مانند `Read/Write` و `User/Supervisor`) به درستی در مدخل‌های جدول صفحه تنظیم شوند.

پرسش 9:

شیوه ی بارگذاری برنامه در حافظه توسط فراخوانی سیستمی `exec` را شرح دهید.

فراخوانی سیستمی `exec` (که پیاده‌سازی آن در `exec.c` قرار دارد) فرآیند بارگذاری یک برنامه جدید در فضای آدرس فرآیند جاری را بر عهده دارد. این فرآیند در چند مرحله کلیدی صورت می‌گیرد:

1. **آماده‌سازی و اعتبارسنجی:** ابتدا، `exec` فایل اجرایی (که مسیر آن در `path` داده شده) را باز کرده و هدر آن را بررسی می‌کند تا مطمئن شود فایل دارای فرمت `ELF` معتبر است. سپس، با استفاده از `setupkvm` یک ساختار جدول صفحه جدید (`pgdir`) برای فرآیند ایجاد می‌کند که هنوز هیچ نگاشت حافظه کاربردی ندارد.

2. **بارگذاری سگمنت‌ها:** تابع بر اساس برنامه‌های بارگذاری (`Program Headers`) در فایل `ELF`، بخش‌های کد و داده برنامه را می‌خواند. برای هر بخش قابل بارگذاری، از تابع `allocuvm` استفاده می‌کند تا فضای آدرس مجازی مورد نیاز در جدول صفحه جدید را تخصیص دهد و اطمینان حاصل کند که صفحات کافی برای محتوای فایل وجود دارد. سپس، تابع `loaduvm` محتوای واقعی داده‌ها را از فایل روی دیسک به فریم‌های فیزیکی نگاشت شده در فضای آدرس مجازی جدید کپی می‌کند. این مرحله شامل اطمینان از این است که آدرس‌های شروع سگمنت‌ها بر مرز صفحات (`PGSIZE`) قرار دارند.

3. **تنظیم پشته کاربر:** پس از بارگذاری سگمنت‌ها، `exec` یک بلوک حافظه جدید را برای پشته کاربر (`User Stack`) در انتهای فضای آدرس تخصیص می‌دهد (با استفاده مجدد از `allocuvm`). همانطور که قبلاً ذکر شد، صفحه اول این ناحیه به عنوان **صفحه محافظ** تنظیم می‌شود. سپس آرگومان‌های خط فرمان (`argv`) و متغیرهای محیطی (در صورت وجود) از فضای کرنل به فضای آدرس کاربر جدید کپی می‌شوند و آدرس‌های آن‌ها در پشته تنظیم می‌گردد.

4. **سوئیچ کردن:** در نهایت، اگر تمام مراحل موفقیت‌آمیز باشد، کرنل جدول صفحه قدیمی فرآیند (`oldpgdir`) را با جدول صفحه جدید (`pgdir`) جایگزین می‌کند (توسط `switchuvm`). همچنین اندازه حافظه فرآیند (`sz<-curproc`) به‌روزرسانی شده و اشاره‌گر دستورالعمل (`EIP`) و اشاره‌گر پشته (`ESP`) در ساختار رجیستر تراپ (`tf<-curproc`) برای شروع اجرای برنامه تنظیم می‌شوند. در این نقطه، فرآیند برای اجرای کد کاربر آماده است و اجرای آن با اولین دستورالعمل برنامه در آدرس `elf.entry` آغاز خواهد شد.

پرسش 10:

در مورد الگوریتم های مورد استفاده در سیستم عامل های ویندوز و لینوکس تحقیق کنید و به طور خلاصه کاربردهای هر کدام را ذکر کنید.

الگوریتم های جایگزینی صفحات در ویندوز (Windows)

ویندوز از یک الگوریتم جایگزینی صفحه پیچیده مبتنی بر Work Set Model استفاده می کند که بر اساس صفحات در حال استفاده فعال (Working Set) هر فرآیند استوار است. هدف اصلی آن حفظ صفحات مورد نیاز فرآیندهای فعال در حافظه و جابه جایی صفحات کم استفاده به دیسک است.

Working Set Threshold: ویندوز برای هر فرآیند یک "مجموعه کاری" (Work Set) شامل صفحات فعال تعریف می کند. اگر تعداد صفحات یک فرآیند از یک آستانه (Threshold) فراتر رود، سیستم ممکن است شروع به Swapping (جابجایی کل فرآیند به دیسک) کند تا حافظه برای فرآیندهای دیگر آزاد شود.

(Modified LRU) / Clock Algorithm: ویندوز از نسخه ای از الگوریتم ساعت (Clock) یا LRU اصلاح شده استفاده می کند. این الگوریتم از بیت های مورد استفاده (Use Bit) و بیت های تغییر یافته (Modified/Dirty Bit) برای تعیین اولویت صفحات برای خروج استفاده می کند. صفحاتی که اخیراً استفاده شده اند (Use=1) یا تغییر یافته اند (Modified=1) اولویت کمتری برای خروج دارند.

تمرکز ویندوز بر تضمین عملکرد قابل قبول (Performance Guarantees) برای برنامه های در حال اجرا با حفظ مجموعه های کاری آنها در حافظه است، در حالی که امکان حذف کامل فرآیندهای بیکار را نیز فراهم می کند.

الگوریتم های جایگزینی صفحات در لینوکس (Linux)

لینوکس از الگوریتم Clock (ساعت) به عنوان پایه، با افزودن مکانیزم های پیچیده برای مدیریت فعالیت صفحات استفاده می کند که آن را شبیه به LRU می کند.

Clock Algorithm (Active/Inactive List): لینوکس صفحات را به دو لیست دسته بندی می کند: لیست فعال (Active List) و لیست غیرفعال (Inactive List).

صفحاتی که اخیراً استفاده شده‌اند به لیست فعال منتقل می‌شوند. هنگامی که نیاز به صفحه جدید است، سیستم از لیست غیرفعال شروع به بررسی می‌کند. صفحاتی که بیت استفاده آن‌ها صفر است از لیست غیرفعال خارج و جایگزین می‌شوند. اگر بیت استفاده 1 باشد، بیت به 0 تغییر کرده و صفحه به انتهای لیست غیرفعال بازگردانده می‌شود (شبیه به یک دور ساعت).

Page Replacement Daemon (kswapd): یک فرآیند کرنل به نام kswapd به طور مداوم و در پس‌زمینه فعال است. این دیمون زمانی که میزان حافظه آزاد به زیر یک آستانه مشخص می‌رسد، صفحات را از لیست غیرفعال انتخاب و به دیسک منتقل می‌کند تا فضای کافی برای درخواست‌های جدید فراهم شود.

“Caches” (صفحات حافظه پنهان): لینوکس بخش بزرگی از حافظه آزاد را به عنوان حافظه پنهان برای فایل‌های دیسک (Page Cache) نگه می‌دارد. این صفحات با اولویت کمتری نسبت به صفحات فرآیندها تعویض می‌شوند.

لینوکس بر بهره‌وری کلی سیستم (Overall System Throughput) تمرکز دارد و سعی می‌کند فضای حافظه را به طور مؤثر بین حافظه پنهان دیسک و صفحات فرآیندها تقسیم کند. الگوریتم Clock تضمین می‌کند که صفحات کمتر استفاده شده اولویت بیشتری برای جایگزینی دارند.

پرسش 11:

مقایسه ی برنامه ها را طبق معیار هایی که خواسته شده انجام دهید و بنویسید آیا طبق آنچه که از الگوریتم ها میدانید این رفتار قابل پیش بینی است یا خیر؟

• ساختار داده جدول صفحه

paging.h برای تعریف ساختارها و ماکروها .

paging.c برای پیاده‌سازی جدول صفحه مرکزی، توابع مدیریت اولیه، و نگاشت‌سازی (جایی که

منطق جایگزینی نیز قرار می‌گیرد) .

sysfile.c برای پیاده‌سازی دو سیستم کال درخواستی.

```
xv6-public-master > C paging.h > ...
1  #ifndef PAGING_H
2  #define PAGING_H
3
4  #include "types.h"
5  #include "spinlock.h"
6
7  #define FRAME_TABLE_SIZE 4
8
9  struct PageTableEntry {
10     char *physical_addr;
11     uint virtual_page_num;
12     int process_id;
13     uint valid;
14     uint access_time;
15     uint load_time;
16     uint usage_count;
17 };
18
19 extern struct PageTableEntry frame_table[FRAME_TABLE_SIZE];
20 extern struct spinlock frame_table_lock;
21
22 void frame_table_init(void);
23 void invalidate_process_pages(int pid);
24 struct PageTableEntry* get_or_load_page(uint va, int pid);
25
26 #endif
27
```

vv6-public-master > C paging.c > select_victim_frame()

```
14
15 void frame_table_init(void) {
16     initlock(&frame_table_lock, "frame_table");
17     for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
18         frame_table[i].valid = 0;
19         frame_table[i].process_id = -1;
20     }
21 }
22
23 void invalidate_process_pages(int pid) {
24     acquire(&frame_table_lock);
25     for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
26         if (frame_table[i].process_id == pid && frame_table[i].valid == 1) {
27             frame_table[i].valid = 0;
28             frame_table[i].process_id = -1;
29         }
30     }
31     release(&frame_table_lock);
32 }
33
34 struct PageTableEntry* get_or_load_page(uint va, int pid) {
35     acquire(&frame_table_lock);
36
37     uint vpn = va / PGSIZE;
38     for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
39         if (frame_table[i].valid == 1 && frame_table[i].process_id == pid && frame_table[i].virtual_page_num == vpn) {
40             frame_table[i].access_time = ticks;
41             release(&frame_table_lock);
42             return &frame_table[i];
43         }
44     }
45
46     int target_frame = -1;
47     for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
48         if (frame_table[i].valid == 0) {
49             target_frame = i;
50             break;
51         }
52     }
53     if (target_frame == -1) {
54         target_frame = select_victim_frame();
55         if (target_frame == -1) {
56             release(&frame_table_lock);
57             return 0;
58         }
59     }
60
61     struct PageTableEntry *pte = &frame_table[target_frame];
62
63     if (pte->valid == 1 && pte->physical_addr) {
64         kfree(pte->physical_addr);
65     }
66
67     pte->physical_addr = kalloc();
68     if (pte->physical_addr == 0) {
69         release(&frame_table_lock);
70         return 0;
71     }
72 }
```

```

449 int
450 sys_pagetable_write(void) {
451     void *va;
452     int value;
453     struct PageTableEntry *pte;
454     struct proc *curproc = myproc();
455
456     if(argptr(0, (void*)&va, sizeof(va)) < 0 || argint(1, &value) < 0)
457         return -1;
458
459     pte = get_or_load_page((uint)va, curproc->pid);
460
461     if (!pte)
462         return -1;
463
464     uint offset = (uint)va % PGSIZE;
465     char *target_pa = pte->physical_addr + offset;
466
467     *(int*)target_pa = value;
468
469     acquire(&frame_table_lock);
470     pte->access_time = ticks;
471     pte->usage_count++;
472     release(&frame_table_lock);
473
474     return 0;
475 }
476
477 int
478 sys_pagetable_read(void) {
479     void *va;
480     int result = 0;
481     struct PageTableEntry *pte;
482     struct proc *curproc = myproc();
483
484     if(argptr(0, (void*)&va, sizeof(va)) < 0)
485         return -1;
486
487     pte = get_or_load_page((uint)va, curproc->pid);
488
489     if (!pte)
490         return -1;
491
492     uint offset = (uint)va % PGSIZE;
493     char *source_pa = pte->physical_addr + offset;
494
495     result = *(int*)source_pa;
496
497     acquire(&frame_table_lock);
498     pte->access_time = ticks;
499     release(&frame_table_lock);
500
501     return result;
502 }
503

```

• الگوریتم های جایگزینی صفحات

این جایگزینی تنها زمانی رخ دهد که جدول صفحه پر باشد (تمام صفحات معتبر باشند). برای پیاده سازی کامل هر چهار الگوریتم، تابع انتخاب قربانی (select_victim_frame) باید پارامتری برای تعیین الگوریتم فعال دریافت کند و ساختار PageTableEntry باید برای همه آن ها کافی باشد.

```

86 int select_victim_frame() {
87     switch (active_page_replacement_algo) {
88         case ALGO_FIFO:
89             uint oldest_time = (uint)-1;
90             for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
91                 if (frame_table[i].load_time < oldest_time) {
92                     oldest_time = frame_table[i].load_time;
93                     victim_idx = i;
94                 }
95             }
96             break;
97
98         case ALGO_LRU:
99             {
100                 uint latest_time = (uint)-1;
101                 for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
102                     if (frame_table[i].access_time < latest_time) {
103                         latest_time = frame_table[i].access_time;
104                         victim_idx = i;
105                     }
106                 }
107             }
108             break;
109
110         case ALGO_LFU:
111             {
112                 uint min_freq = (uint)-1;
113                 int best_victim = -1;
114
115                 for (int i = 0; i < FRAME_TABLE_SIZE; i++) {
116                     if (frame_table[i].usage_count < min_freq) {
117                         min_freq = frame_table[i].usage_count;
118                         best_victim = i;
119                     } else if (frame_table[i].usage_count == min_freq) {
120                         if (frame_table[i].load_time < frame_table[best_victim].load_time) {
121                             best_victim = i;
122                         }
123                     }
124                 }
125                 victim_idx = best_victim;
126             }
127             break;
128
129         case ALGO_CLOCK:
130             {
131                 static int clock_hand = 0;
132                 int start_hand = clock_hand;
133
134                 do {
135                     struct PageTableEntry *pte = &frame_table[clock_hand];
136
137                     if (pte->access_time < ticks - 1) {
138                         victim_idx = clock_hand;
139                         clock_hand = (clock_hand + 1) % FRAME_TABLE_SIZE;
140                         release(&frame_table_lock);
141                         return victim_idx;
142                     }
143
144                     pte->access_time = ticks;
145                     clock_hand = (clock_hand + 1) % FRAME_TABLE_SIZE;
146                 } while (clock_hand != start_hand);
147
148                 victim_idx = start_hand;
149             }
150             break;
151
152         default:
153             victim_idx = -1;
154             break;
155     }
156 }

```

- معیارهای ارزیابی الگوریتمها

$$\text{Hit Ratio} = (\text{Hit Count} / \text{Total Accesses}) \times 100$$

```
504 int
505 sys_getpagestats(void) {
506     struct proc *curproc = myproc();
507     curproc->stats.end_time_ticks = ticks;
508     uint total_time = curproc->stats.end_time_ticks - curproc->stats.start_time_ticks;
509
510     cprintf("--- Page Replacement Algorithm Performance Report ---\n");
511
512     struct page_alg_stats algs[] = {
513         curproc->stats.fifo_stats,
514         curproc->stats.lru_stats,
515         curproc->stats.lfu_stats,
516         curproc->stats.clock_stats
517     };
518
519     char *names[] = {"FIFO", "LRU", "LFU", "Clock"};
520
521     for (int i = 0; i < 4; i++) {
522         int hits = algs[i].hit_count;
523         int total = algs[i].total_accesses;
524         double ratio = (total > 0) ? (double)hits * 100.0 / total : 0.0;
525
526         cprintf("Algorithm: %s\n", names[i]);
527         cprintf(" Hit Count: %d\n", hits);
528         cprintf(" Hit Ratio: %.2f%%\n", ratio);
529         cprintf(" Execution Time (Ticks): %u\n", total_time);
530     }
531
532     cprintf("-----\n");
533
534     return 0;
535 }
536
```

برنامه های سطح کاربر

```
xv6-public-master > C test1.c > LARGE_ARRAY_SIZE
1  #include "user.h"
2  #include "types.h"
3  #include "time.h"
4  #include "paging.h"
5  #include "paging.c"
6
7  #define LARGE_ARRAY_SIZE (4 * 1024 * 1024)
8  int *data = (int *) 0x10000000;
9
10 void test_memory_access() {
11     int i;
12     int count = 0;
13     int target_pages = 10;
14
15     for (i = 0; i < LARGE_ARRAY_SIZE; i += 4096) {
16         if (count >= target_pages) break;
17
18         data[i / sizeof(int)] = i;
19         count++;
20     }
21 }
22
23 int main(int argc, char *argv[]) {
24     uint start_time = getticks();
25     test_memory_access();
26     uint end_time = getticks();
27     printf(1, "Program finished. Execution Time: %u ticks\n", end_time - start_time);
28     getpagestats();
29     exit();
30     return 0;
31 }
32
```


xv6-public-master > C test2.c > main(int, char * [])

```
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4  #include "fs.h"
5  #include "paging.h"
6  #include "paging.c"
7
8  #define NUM_PAGES 5
9  #define NUM_RUNS 4
10 #define PAGE_SIZE 4096
11
12
13 #define SYS_PAGETEST 300
14
15
16 void access_page(char* base_addr, int page_index) {
17     base_addr[page_index * PAGE_SIZE] = (char)(page_index + '0');
18 }
19
20 void
21 main(int argc, char *argv[])
22 {
23     char *memory_block;
24     memory_block = sbrk(NUM_PAGES * PAGE_SIZE);
25     if (memory_block == (char *)0xffffffff) {
26         printf(2, "sbrk failed\n");
27         exit();
28     }
29
30     memset(memory_block, 0, NUM_PAGES * PAGE_SIZE);
31     for (int run = 0; run < NUM_RUNS; run++) {
32         for (int access = 0; access < NUM_PAGES; access++) {
33             int page_to_access = access % NUM_PAGES;
34             access_page(memory_block, page_to_access);
35         }
36     }
37     getpagstats();
38     exit();
39     return 0;
40 }
41
```

xv6-public-master > C test3.c > main(int, char *[])

```
1  #include "types.h"
2  #include "user.h"
3  #include "fs.h"
4  #include "paging.h"
5  #include "paging.c"
6
7
8  #define NUM_HOT_PAGES 3
9  #define NUM_COLD_PAGES 5
10 #define TOTAL_PAGES (NUM_HOT_PAGES + NUM_COLD_PAGES)
11 #define OUTER_RUNS 50
12 #define INNER_ACCESSES 100
13
14 void access_memory(char* base_addr, int page_index) {
15     base_addr[page_index * 4096] = (char)(page_index + '0');
16 }
17
18 void
19 main(int argc, char *argv[])
20 {
21     char *memory_block;
22     memory_block = sbrk(TOTAL_PAGES * 4096);
23     if (memory_block == (char *)0xffffffff) {
24         printf(2, "sbrk failed to allocate memory.\n");
25         exit();
26     }
27     memset(memory_block, 0, TOTAL_PAGES * 4096);
28     uint startTime = rdtsc();
29     int total_recorded_accesses = 0;
30     for (int run = 1; run <= OUTER_RUNS; run++) {
31         for (int i = 1; i <= INNER_ACCESSES; i++) {
32             if (total_recorded_accesses < OUTER_RUNS * INNER_ACCESSES) {
33                 access_memory(memory_block, 0); total_recorded_accesses++;
34             }
35             if (total_recorded_accesses < OUTER_RUNS * INNER_ACCESSES) {
36                 access_memory(memory_block, 1); total_recorded_accesses++;
37             }
38             if (total_recorded_accesses < OUTER_RUNS * INNER_ACCESSES) {
39                 access_memory(memory_block, 2); total_recorded_accesses++;
40             }
41
42             for (int j = 0; j < NUM_COLD_PAGES; j++) {
43                 if (total_recorded_accesses < OUTER_RUNS * INNER_ACCESSES) {
44                     access_memory(memory_block, NUM_HOT_PAGES + j);
45                     total_recorded_accesses++;
46                 } else {
47                     break;
48                 }
49             }
50
51             if (total_recorded_accesses > (run * INNER_ACCESSES) && run < OUTER_RUNS) break;
52         }
53     }
54
55     uint endTime = rdtsc();
56     uint totalTime = endTime - startTime;
57     getpagestats();
58     exit();
59     return 0;
60 }
```


xv6-public-master > C test4.c > main()

```
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4  #include "fs.h"
5  #include "paging.h"
6  #include "paging.c"
7
8  #define NUM_PAGES 5
9  #define OUTER_RUNS 100
10
11 int
12 main() {
13     int pages[NUM_PAGES];
14
15     for (int i = 0; i < NUM_PAGES; i++) {
16         pages[i] = i;
17     }
18
19     for (int run = 0; run < OUTER_RUNS; run++) {
20         pages[0];
21         pages[1];
22         pages[2];
23         pages[3];
24         pages[0];
25         pages[1];
26         pages[4];
27     }
28
29     getpagestats();
30     exit();
31     return 0;
32 }
33
```

